Mirea Sacco

ICT1018 – Data Structures and Algorithims

10/05/2025

Coursework Documentation

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as "the unacknowledged use, as one's own work, of work of another person, whether or not such work has been published" (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

I / We*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / our* work, except where acknowledged and referenced.

I / We* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

Mirea Sacco	
Student Name	Signature
Student Name	Signature
Student Name	Signature
Student Name	Signature

ICT1018	Coursework 2025
Course Code	Title of work submitted

08/05/2025

Date

Statement of Completion

Question 1 – Attempted and works well

Question 2 – Attempted and works well

Question 3 – Attempted and works well

Question 4 – Attempted and works well

Question 5 – Attempted and works well

Question 6 – Attempted and works well

Question 7 – Attempted and works well

Question 8 – Attempted and works well

Question 9 – Attempted and works well

Question 10 – Attempted and works well

Question 11 – Attempted and works well

Question 12 – Attempted and works well

Generative AI disclosure

Promts used:

- I need help with my quick sort algorithim in python because it is not returning the correct list, help me with the while statements
- Explain the Newton Raphson method with example?
- Give me the equation to use to find the square root by Newton Raphson
- Is my factorial function correct?
- How to implement sine and cosine using taylor series?
- Why is my cosine and sine approximation returning incorrect values?
- What is a memory efficient way to remove duplicates in a list?
- How to remove specific value from a list using a variable?
- How to check if a value exists in a list?
- How to use a for loop that increments by 2?
- Is halfing a list by 2 recursively a good way to find the largest number in a list?
- How to calculate the numbers of the fibonacci sequence?
- What is an ADT stack?
- What is reverse polish notation?
- What does split() do?
- How to use elif in an if statement?
- How to check if a string is one of several operators?

Question 1

Create two integer arrays A and B. The two arrays must contain at least 256 elements but must be of unequal size. Populate both arrays with randomly generated integers between 0 and 1024. Sort array A using Shell Sort and array B using Quick Sort.

I generated 2 lists A and B containing random integers from 0 to 1024 using python's function random.randint().

The shell sort algorithim compairs elements with a specific gap away from eachother, starting from half the length of the list. This comparison will iteritively repeated, halfing the gap each time until it is 1. The algorithim checks if 2 elements are sorted by keeping a boolean variable and if the left element is bigger than the right element, they will be swapped.

The quick sort will take the first term in the sequence as a pivot, the list will be partioned into the elements smaller than the pivot and the elements larger than the pivot. The partition function partition() handles the logic applied to the sublists created.

```
import random
A = [random.randint(0, 1024) for _ in range(257)]
B = [random.randint(0, 1024) for _ in range(300)]

def shell_sort(arr):
    n = len(arr) // 2
    flag = True
    i = 0
    while n>=1 and flag:
        flag = False
        for i in range(len(arr) - n):
            if arr[i] > arr[i+n]:
                temp = arr[i]
                arr[i] = arr[i+n]
                arr[i+n] = temp
                flag = True
        if n>1:
            n = n // 2
    return arr

def quick_sort(arr, first, last):
    if first < last:
        pivot_pos = partition(arr, first, last)
        quick_sort(arr, first, pivot_pos-1)
        quick_sort(arr, pivot_pos+1, last)

def  partition(arr, first, last):
    pivot = arr[first]
    left = first
    right = last

    while left<right:
```

```
        while left <= last and arr[left] <= pivot:
            left += 1
        while right >= first and arr[right] > pivot:
            right -= 1
        if left<right:
            temp = arr[left]
            arr[left] = arr[right]
            arr[right] = temp

    temp = pivot
    arr[first] = arr[right]
    arr[right] = pivot
    return right
A = shell_sort(A)
quick_sort(B, 0 ,len(B)-1)
```

Testing

Lists A and B were first printed unsorted, then they were printed again ones they were sorted using the algorithim created for shell sort and quick sort. For the sake of testing the arrays were populated with 30 elements so it would be possible to compare lists. This code used for testing was added to the program:

```
print('unsorted A', A)
A = shell_sort(A)
print('sorted A', A)
print('unsorted B', B)
quick_sort(B, 0 ,len(B)-1)
print('sorted B', B)
```

The output for this program is:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

mir@mirealaptop:~/helloworld$ /bin/python3 "/home/mir/helloworld/DSA coursework/1,2.py"
unsorted A [271, 335, 411, 993, 334, 838, 600, 686, 1018, 721, 382, 449, 637, 975, 102, 359, 871, 399, 121, 54, 801, 697, 429, 55, 518, 922, 184, 904, 81, 952]
sorted A [54, 55, 81, 102, 121, 184, 271, 334, 335, 359, 382, 399, 411, 429, 449, 518, 600, 637, 686, 697, 721, 801, 838, 871, 904, 922, 952, 975, 993, 1018]
unsorted B [348, 15, 686, 235, 808, 710, 575, 195, 967, 749, 2, 896, 930, 169, 656, 506, 16, 937, 20, 700, 166, 681, 250, 84, 979, 230, 263, 322, 88, 935]
sorted B [2, 15, 16, 20, 84, 88, 166, 169, 195, 230, 235, 250, 263, 322, 348, 506, 575, 656, 681, 686, 700, 710, 749, 808, 896, 930, 935, 937, 967, 979]
```

Question 2

Take the two sorted arrays A and B from Question 1 above and merge into a new array C. You must do this in linear time. Note that the size of C must be size of A plus size of B. Use the merging technique used in Merge Sort.

The merge function creates pointers on the first element of the sorted lists A and B, takes the smaller one and appends it to the array called result. The pointer to the smaller number will be incremented to point to the next number, this will be repeated until one list is empty and the rest of the elements in the other list will be appended to the result list.

This following code was written in the same file as the code in question 1:

```
def merge(arr1, arr2):
   l1 = len(arr1)
   l2 = len(arr2)
   result = []
   i = j = 0

   while i<l1 and j<l2:
      if arr1[i]<arr2[j]:
         result.append(arr1[i])
         i += 1
      else:
         result.append(arr2[j])
         j += 1

   result.extend(arr1[i:])
   result.extend(arr2[j:])
   return result
```

Testing

Sorted lists A and B were merged using the function merge(). Again, for the sake of testing lists A and B will have 30 elements in them. The length of lists A, B and C are printed, and the expected length of list C is 60(=30+30). This code used for testing was added to the program:

```
print('size of A =', len(A))
print('sorted A', A)
print('size of B =', len(B))
print('sorted B', B)
C = merge(A, B)
print('size of C =', len(C))
print('C', C)
```

The output for this program is:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

mir@mirealaptop:~/helloworld$ /bin/python3 "/home/mir/helloworld/DSA coursework/1,2.py"
size of A = 30
sorted A [10, 50, 65, 76, 86, 89, 152, 210, 215, 224, 243, 256, 318, 370, 484, 550, 567, 579, 592, 629, 674, 701, 728, 812, 861, 861, 868, 887, 936, 944]
size of B = 30
sorted B [50, 86, 194, 198, 230, 284, 287, 302, 318, 325, 364, 384, 418, 434, 447, 462, 507, 509, 581, 633, 642, 685, 688, 704, 839, 904, 906, 942, 951, 1000]
size of C = 60
C [10, 50, 50, 65, 76, 86, 86, 89, 152, 194, 198, 210, 215, 224, 230, 243, 256, 284, 287, 302, 318, 318, 325, 364, 370, 384, 418, 434, 447, 462, 484, 507, 509, 550, 567, 579, 581,
592, 629, 633, 642, 674, 685, 688, 701, 704, 728, 812, 839, 861, 861, 868, 887, 904, 906, 936, 942, 944, 951, 1000]
```

Question 3

Let A be an array of n elements (that is, the elements of A are A[0], . . . , A[n−1]). An element A[i] is called extreme if the following conditions hold regarding A[i].

• A[i] is not the first nor the last element of A. That is, 0 < i < n – 1 and either A[i − 1] < A[i] > A[i + 1] or A[i − 1] > A[i] < A[i + 1]. For example, the extreme points of an array [0, 5, 3, 6, 8, 7, 15, 9] are 5, 3, 8, 7, 15.

Write an algorithm that prints the extreme points of the given array. If there are no extreme points, the algorithm prints "SORTED". Do you agree that an array has no extreme points if and only if it is sorted? Explain your answer.

A for loop starting from the second element was inplimented to check each element for the condition given, if the element met the criteria for it to be extreme, it was appended to a list called extreme.

I agree that an array has no extreme points only if it is sortedeither in acsending or decsending order. This is becuase the conditions check if the element getting checked has a value less or more than both the neighbouring values which would make it an extreme. In a sorted array the values consecutively get smaller or bigger so it does not have extremes.

```
A = []
n = len(A)

extreme = []
for i in range(n-2):
   i += 1
   if ((A[i - 1] < A[i] and  A[i] > A[i + 1]) or (A[i - 1] > A[i] and A[i] < A[i + 1])):
      extreme.append(A[i])

if len(extreme) == 0:
   print('SORTED')
else:
   print(extreme)
```

Testing

After implementing the algorithim, taking the example given in the question, the algorithim was tested and the result expected was 5, 3, 8, 7, 15.

```
A = [0, 5, 3, 6, 8, 7, 15, 9]
```

The output of the program was (as expected):

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

mir@mirealaptop:~/helloworld$ /bin/python3 "/home/mir/helloworld/DSA coursework/2.py"
[5, 3, 8, 7, 15]
```

This list was also given to the program:

```
A = [1, 6, 7, 10, 14, 16]
```

The output of the program was (as expected):

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

mir@mirealaptop:~/helloworld$ /bin/python3 "/home/mir/helloworld/DSA coursework/3.py"
SORTED
```

Question 4

Write a program that, given a list of integers, finds all 2-pairs of integers that have the same product. A 2-pair is 2 distinct pairs of integers ((a,b),(c,d)) where a X b = c X d and a ≠ b ≠ c ≠ d. The range of integers in the list should be from 1 to 1024.

A list 'A' was populated with 100 intigers ranging from 1 and 1024. A dictionary called product_map was used to store the resultant products of all the unique pairs of A and all the pairs that produce the same product. The dictionary was structured as {product: [(a_1 x b_1), (a_2 x b_2), ...] }.

The list final was created so that it will hold all the 2-pairs. A for loop was used to iterate through the items in a dictionary and append each 2 pairs from the

```
import random

A = [random.randint(1, 1024) for _ in range(100)]

from collections import defaultdict

product_map = defaultdict(list)

for  i in range(len(A)):
   for j in range(i+1, len(A)):
      a = A[i]
      b = A[j]
      product = a * b

      if a != b:
         product_map[product].append((a, b))

final = []
for product, pairs in product_map.items():
   for i in range(len(pairs)):
      for j in range(i+1, len(pairs)):
         a, b = pairs[i]
         c, d = pairs[j]

         test = set([a, b, c, d])

         if len(test) == 4:
            final.append([product, (a, b), (c, d),])

for i in final:
   print(i)
```

Testing

The output printed all valide 2-pairs with the same product in a list.

The output of the program was(as expected):

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

mir@mirealaptop:~/helloworld$ /bin/python3 "/home/mir/helloworld/DSA coursework/4.py"
[234696, (889, 264), (231, 1016)]
[266475, (969, 275), (285, 935)]
[266475, (969, 275), (285, 935)]
[113430, (285, 398), (995, 114)]
[110580, (285, 388), (114, 970)]
[7980, (285, 28), (798, 10)]
[72848, (628, 116), (232, 314)]
[20856, (79, 264), (33, 632)]
[20856, (79, 264), (33, 632)]
[18328, (79, 232), (29, 632)]
[22120, (79, 280), (632, 35)]
[26334, (231, 114), (33, 798)]
[26334, (231, 114), (33, 798)]
[26796, (231, 116), (33, 812)]
[26796, (231, 116), (33, 812)]
[78585, (507, 155), (195, 403)]
[47336, (97, 488), (122, 388)]
[27160, (97, 280), (970, 28)]
[11252, (97, 116), (29, 388)]
[7656, (264, 29), (232, 33)]
[7656, (264, 29), (232, 33)]
[217536, (264, 824), (309, 704)]
[9240, (264, 35), (33, 280)]
[9240, (264, 35), (280, 33)]
[21576, (29, 744), (124, 174)]
[14152, (29, 488), (122, 116)]
[8120, (29, 280), (232, 35)]
[8120, (29, 280), (812, 10)]
[8120, (232, 35), (812, 10)]
[688160, (748, 920), (935, 736)]
[115940, (748, 155), (935, 124)]
[26180, (748, 35), (935, 28)]
[386060, (398, 970), (995, 388)]
[45240, (232, 195), (174, 260)]
[4872, (232, 21), (174, 28)]
[80352, (837, 96), (124, 648)]
[72480, (151, 480), (96, 755)]
[72480, (151, 480), (96, 755)]
[127400, (490, 260), (280, 455)]
[193440, (744, 260), (403, 480)]
```

Question 5

Write a program that uses an ADT Stack to evaluate arithmetic expressions in RPN format.

The contents of the stack should be displayed on the screen during evaluation. The allowed arithmetic operators are +, -, x, and /.

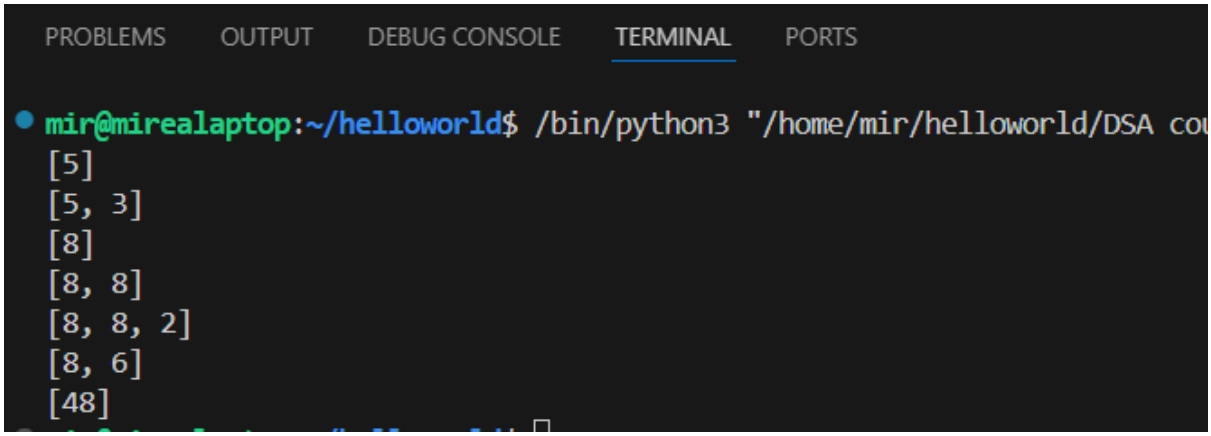
When given an input, the program will first split the items and ut them into a list called inp_list. It will iterate though the elements of this list and append it to the stack, then if one of them are an operation it will make the calculation and append it to the stack instead of the numbers used from the stack. The stack is printed each iteration to follow what happens during the program.

```
inp = "5 3 + 8 2 - x"
inp_list = inp.split()
stack = []
operations = ['+', '-', 'x', '/']
for i in inp_list:
   if i not in operations:
      stack.append(int(i))
   else:
      num2 = stack.pop()
      num1 = stack.pop()
      if i =='+':
         ans = num1 + num2
      elif i =='-':
         ans = num1 - num2
      elif i =='/':
         ans = num1 / num2
      elif i =='x':
         ans = num1 * num2
      stack.append(ans)
   print(stack)
```

Testing

The test input '5 3 + 8 2 - x' was given to the program as the value for the variable inp. The expected output is 48 since (5 + 3) * (8 - 2) = 8 * 6 = 48.

The output of the program was (as expected):



Question 6

Write a Boolean function that checks if a number is prime. Also implement the Sieve of Eratosthenes algorithm – two separate algorithms. Explain any optimizations made.

is_prime() function: The function checks if the input number is divided by any number fbetween 2 and square root n. It it gives a remainder of zero, the function returns false. If none of the numbers up to the given one divide it exactly, it will return true.

For this function, an optimisation was applied so that the number of iterations will be reduced when a number is prime. The loop would only iterate to numbers up to square root n, since any number that is not prime will already have a factor between 2 and square root n.

sieve() function: this function, first, makes a list of the numbers from 2 to the number n. It then iterates through each number and checks which numbers are dicisible by it with a remainder or zero. These numbers are removed from the list and it moves on the the next number.

```
def is_prime(num):
    i=2
    while i<=(num**0.5):
        if num%i == 0:
            return False
        i += 1
    return True

def sieve(n):
    list = []
    for i in range(2,n+1):
        list.append(i)

    for i in list:
        for j in list:
            if i != j and j%i==0:
                list.remove(j)

    return list
```

Testing

This code was used to test the program, it was written after the algorithim written above. The expected result is that 50 is not prime, 97 is prime and all the prime numbers before 100 are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97

```
print(is_prime(50))
print(sieve(100))
```

The output of the program was (as expected):

```
mir@mirealaptop:~/helloworld$ /bin/python3 "/home/mir/helloworld/DSA coursework/6.py"
False
True
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
```

Question 7

*Write a program that generates the **Collatz sequence** of all the integers in the range 2 to 512. The program will store the generated sequences in a CSV file.*

A function for the Collatz sequence was created. It returns the sequence for the number passed to the function. A CSV file called collatz_seq was created. Using a for loop, the numbers from 2 to 512 were passed to the Collatz function and each resulting sequence was written in the CSV file.

```
import csv
def collatz_seq(n):
    seq = str(n)
    while n>1:
        if n % 2 == 0:
            n = n // 2
        else:
            n = (n * 3) + 1
        seq = seq + ' ' + str(n)
    return seq

with open('collatz_seq.csv', 'w') as collatz:

    fieldnames = ['integer','sequence']

    writer = csv.DictWriter(collatz, fieldnames = fieldnames)

    writer.writeheader()

    for i in range(2, 513):
        numbers = collatz_seq(i)
        writer.writerow({'integer' : i, 'sequence' : numbers})
```

Testing

After running the program, it was expected that a CSV file would be created and all the sequences would be written in it.

The file created was(as expected):

7.py U collatz_seq.csv U

collatz_seq.csv

```csv
integer,sequence
2,2 1
3,3 10 5 16 8 4 2 1
4,4 2 1
5,5 16 8 4 2 1
6,6 3 10 5 16 8 4 2 1
7,7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
8,8 4 2 1
9,9 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
10,10 5 16 8 4 2 1
11,11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
12,12 6 3 10 5 16 8 4 2 1
13,13 40 20 10 5 16 8 4 2 1
14,14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
15,15 46 23 70 35 106 53 160 80 40 20 10 5 16 8 4 2 1
16,16 8 4 2 1
17,17 52 26 13 40 20 10 5 16 8 4 2 1
18,18 9 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
19,19 58 29 88 44 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
20,20 10 5 16 8 4 2 1
21,21 64 32 16 8 4 2 1
22,22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
23,23 70 35 106 53 160 80 40 20 10 5 16 8 4 2 1
24,24 12 6 3 10 5 16 8 4 2 1
25,25 76 38 19 58 29 88 44 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
26,26 13 40 20 10 5 16 8 4 2 1
27,27 82 41 124 62 31 94 47 142 71 214 107 322 161 484 242 121 364 182 91 274 137 412 206 103 310 155 466 233 700 350 175 526 263 790 395 1186
28,28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
29,29 88 44 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
30,30 15 46 23 70 35 106 53 160 80 40 20 10 5 16 8 4 2 1
31,31 94 47 142 71 214 107 322 161 484 242 121 364 182 91 274 137 412 206 103 310 155 466 233 700 350 175 526 263 790 395 1186 593 1780 890 44
32,32 16 8 4 2 1
33,33 100 50 25 76 38 19 58 29 88 44 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
34,34 17 52 26 13 40 20 10 5 16 8 4 2 1
35,35 106 53 160 80 40 20 10 5 16 8 4 2 1
36,36 18 9 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
37,37 112 56 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
```

...

```csv
501,501 1504 752 376 188 94 47 142 71 214 107 322 161 484 242 121 364 182 91 274 137 412 206 103 310 155 466 233 700 350 175 526 263 790 395 1
502,502 251 754 377 1132 566 283 850 425 1276 638 319 958 479 1438 719 2158 1079 3238 1619 4858 2429 7288 3644 1822 911 2734 1367 4102 2051 61
503,503 1510 755 2266 1133 3400 1700 850 425 1276 638 319 958 479 1438 719 2158 1079 3238 1619 4858 2429 7288 3644 1822 911 2734 1367 4102 205
504,504 252 126 63 190 95 286 143 430 215 646 323 970 485 1456 728 364 182 91 274 137 412 206 103 310 155 466 233 700 350 175 526 263 790 395
505,505 1516 758 379 1138 569 1708 854 427 1282 641 1924 962 481 1444 722 361 1084 542 271 814 407 1222 611 1834 917 2752 1376 688 344 172 86
506,506 253 760 380 190 95 286 143 430 215 646 323 970 485 1456 728 364 182 91 274 137 412 206 103 310 155 466 233 700 350 175 526 263 790 395
507,507 1522 761 2284 1142 571 1714 857 2572 1286 643 1930 965 2896 1448 724 362 181 544 272 136 68 34 17 52 26 13 40 20 10 5 16 8 4 2 1
508,508 254 127 382 191 574 287 862 431 1294 647 1942 971 2914 1457 4372 2186 1093 3280 1640 820 410 205 616 308 154 77 232 116 58 29 88 44 22
509,509 1528 764 382 191 574 287 862 431 1294 647 1942 971 2914 1457 4372 2186 1093 3280 1640 820 410 205 616 308 154 77 232 116 58 29 88 44 2
510,510 255 766 383 1150 575 1726 863 2590 1295 3886 1943 5830 2915 8746 4373 13120 6560 3280 1640 820 410 205 616 308 154 77 232 116 58 29 88
511,511 1534 767 2302 1151 3454 1727 5182 2591 7774 3887 11662 5831 17494 8747 26242 13121 39364 19682 9841 29524 14762 7381 22144 11072 5536
512,512 256 128 64 32 16 8 4 2 1
```

Question 8

*Write a program that finds an approximation to the **square root** of a given number **n** using an iterative numerical method such as the **Newton-Raphson Method**.*

I got the equation for Newton Raphson. This equation was used to write the newt_raph function. The variable x is updated each iteration of the for loop to a more accurate answer. 10 approximations were chosen since it would be closer to the actual answer.

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{\eta}{x_n}\right)$$

```
def newt_raph(n):
    x = n
    for i in range(10):
        x = 0.5 * (x + (n/x))
    return x
```

Testing

To test this function the following numbers were passed through the function:

```
print(newt_raph(45))
print(newt_raph(36))
print(newt_raph(70))
```

The actual answers are: $\sqrt{45} \approx 6.708204$, $\sqrt{36} = 6.000000$, $\sqrt{70} \approx 8.366600$

The output of the program was(as expected):

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL
mir@mirealaptop:~/helloworld$ /bin/python3 "
6.708203932499369
6.0
8.366600265340756
mir@mirealaptop:~/helloworld$
```

These numbers are very close to the actual answers

Question 9

Write a program that, given an array of integers, finds all integers in the array that are repeated more than once. Try to find a fast and memory-efficient way of doing this.

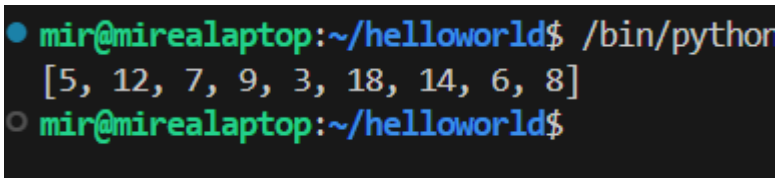
This program checks from repeat numbers in the list A. For each number in the list, if it was not seen before it will be added to the list called seen. If it was already seen and not in the repeats list, it will be appended to the repeats list. The repeats list will be printed, this list will contain all the numbers appear more than once in the list.

```
A = []

seen = []
repeats = []

for num in A:
   if num in seen:
      if num not in repeats:
         repeats.append(num)
   else:
      seen.append(num)

print(repeats)
```

Testing

The list or numbers 12, 5, 7, 9, 5, 12, 18, 21, 7, 3, 9, 15, 3, 14, 18, 6, 5, 8, 9, 11, 14, 2, 6, 10, 7, 13, 8, 17, 3, 5, 20, 12, 9 was given to the program as A. The repeated numbers are underlined (5, 12, 7, 9, 3, 18, 14, 6, 8)

The output of the program was (as expected):



Question 10

*Write a **recursive** function that finds the largest number in a given list of integers.*

The function find_max uses a divide and conquer technique. Until the sub lists has 1 element, the list will be split into two halves. When the base case is reached, the single value in the list is returned as the current maximum. That value will be compared to the next one and return the maximum between them. This will happen recursively until all the sub lists are checked and the function goes out of recursion.

```
def find_max(list):

    if len(list) == 1:
        return list[0]

    leng = len(list)//2
    max1 = find_max(list[leng:])
    max2 = find_max(list[:leng])

    if max1 > max2:
        largest = max1
    else:
        largest = max2

    return largest

nums = []
print(find_max(nums))
```

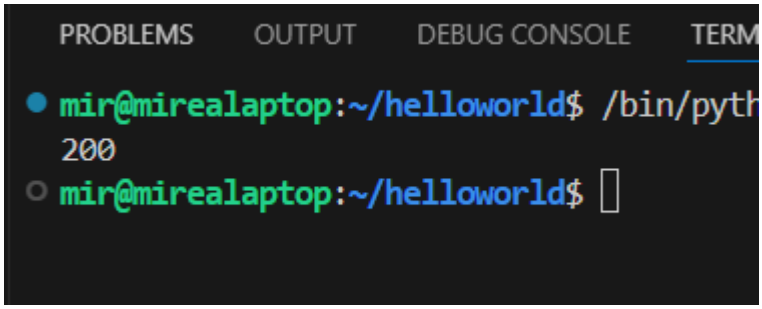
Testing

The function is given the list:

```
nums = [34, 35, 120, 100, 20, 200]
print(find_max(nums))
```

The expected maximum returned is 200.

The output of the program was (as expected):



Question 11

*Write a function that computes **cosine** or **sine** by taking the first **n** terms of the appropriate (Maclaurin?) series expansion. Be careful with computing large factorials.*

I defined a function to calculate the factorial of a number. I also implement a function called addition() to calculate the terms of the series: $\text{ans} = (-1)^{\frac{i-\text{start}}{2}} \frac{x^i}{i!}$ The start value for sine is 1 and 0 for cosine, since the series start from x^1 and cosine from x^0. 10 terms of the series were used since they give a reasonably accurate result. The input angle was changed from degrees to radians since the Maclauren series works with radians.

```
import math

def factorial(n):
   ans = 1
   for x in range(1, n+1):
      ans = x * ans
   return ans

def addition(x, start):
   ans=0
   for i in range(start, 20, 2):
      sign = (-1) ** ((i - start) // 2)
      ans = ans + (sign * (x**i)/factorial(i))
   return ans

def sine(x):
   x = math.radians(x)
   ans = addition(x, 1)
   return ans

def cosine(x):
   x = math.radians(x)
   return addition(x, 0)
```

Testing

The following values were used to test the sine and cosine function and compared to calculator answers.

```
print(sine(90), ',  ',cosine(90))
```

sine(90) = 1 and cosine(90)= 0

The output of the program was (as expected):

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

mir@mirealaptop:~/helloworld$ /bin/python3 "/home/mir/helloworld/DSA coursework/11.py"
1.0 ,   -3.3769215522516056e-15
mir@mirealaptop:~/helloworld$
```

the approximation for cosine(90) is a small number which is very close to zero

Question 12

*Write a function that returns the sum of the first **n** numbers of the **Fibonacci sequence** (Wikipedia). The first 2 numbers in the sequence are 1,1, ...*

The function fibonacci() generates the first n numbers in the Fibonacci sequence. Once the sequence is generated using a for loop that sums up the previous 2 numbers in the sequence and appending the result to the list. Once all the sequence is generated, the function will return the sum of the numbers in the sequence.

```
def fibonacci(n):
    fib = [1, 1]
    for i in range(2, n):
        fib.append(fib[i-2] + fib[i-1])
    return sum(fib)
```

Testing

20 was passed to the function and the expected result is 17710(this was calculated manually on a calculator and also googled). Also, the sequence was printed to make sure that there are 20 elements.

```
print(fibonacci(20))
```

The output of the program was (as expected):

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS
mir@mirealaptop:~/helloworld$ /bin/python3 "/home/mir/helloworld/DSA coursework/12.py"
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765]
17710
mir@mirealaptop:~/helloworld$
```

there are 20 elements and the answer is as expected